Session 1 — Review Q&A

This document captures your answers from the Session 1 review, the verdict on each, and the complete correct answer where your response was partial. Read the complete answers once — then put this away and answer the questions again from memory before Session 2.

Q1 Context window	Partial pass
Q2 Temperature + model tier	Full pass
Q3 Streaming HTTP	Partial pass
Q4 Biggest model fallacy	Full pass
Q5 Model uncertainty (bonus)	Not attempted — answer provided
Q6 Temperature 0 follow-up	Strong pass

Q1. What is a context window, and why does it force you to chunk documents in RAG?

YOUR ANSWER

It is like a RAM where a limited amount of text fits into the window. Chunk documents is forced because of this threshold size of each window.

PARTIAL PASS RAM analogy correct. Chunking explanation restates the problem without explaining the solution.

WHAT WAS MISSING

Your answer described the constraint but not the solution logic. Saying 'forced because of the threshold' just rephrases the problem. The answer needed to explain what chunking actually achieves.

COMPLETE ANSWER

The context window is the model's working memory — everything in one API call (system prompt, conversation, document, response) must fit inside it. A 500-page legal document cannot fit in a single call.

So instead of sending the whole document, you pre-process it: cut it into 300-token chunks, convert each chunk into a vector embedding, and store them. At query time, you retrieve only the 5 most relevant chunks and send those to the model — a tiny slice that fits comfortably.

Key insight: chunking is not just about size. It is about selecting which pieces of a large document are worth sending for a specific query. That selection step — retrieve the relevant slice, not the whole thing — is what RAG actually does.

Q2. NestJS endpoint classifying support tickets into 5 categories. Temperature and model tier?

YOUR ANSWER

I would use temperature 0 for certainty. Temperature 1 is for randomness and best suited for creative writing. I would also use the small model tier for this.

PASS Full pass. Temperature reasoning correct. Bonus credit for unprompted model tier selection.

COMPLETE ANSWER

Temperature 0 — correct. Classification must be deterministic. The same ticket must always get the same category. Sampling randomness at temperature 1 would produce inconsistent categories on identical inputs, which is a silent data bug.

Small model tier — correct. A classification task with 5 fixed categories is well within what an 8B model handles reliably. At scale (thousands of tickets per day) the cost difference between small and mid-tier is significant and entirely unjustified for this task.

Q3. What has to be true about your HTTP response for streaming to work?

YOUR ANSWER

It would need to stream the response and not just do res.send. Although I am not sure about the syntax.

PARTIAL PASS Correct direction. Missing the specific mechanism — why res.send cannot work.

WHAT WAS MISSING

You identified that something different is needed but did not explain the fundamental difference: a normal response closes the connection immediately, streaming requires the connection to stay open.

COMPLETE ANSWER

A normal JSON endpoint works like this: the server waits until the entire response is ready, sends it as one complete payload, then closes the connection. res.json() and res.send() both do this — they finalise and close immediately.

Streaming requires the connection to stay open for the entire duration of token generation. The server sends data in small chunks as each token arrives. This is called Server-Sent Events (SSE) or chunked transfer encoding.

In NestJS, this means returning an Observable or setting the response header manually: res.setHeader('Content-Type', 'text/event-stream') and then writing chunks with res.write() instead of res.send(). You will build this in Session 7.

One-line summary: the connection staying open is the fundamental difference. res.json() cannot stream because it closes the connection on first write.

Q4. A colleague says: just use the biggest model, it is more accurate. When is this wrong?

YOUR ANSWER

It consumes more tokens and takes more time. Not everything is one size fits all. For classification and structured output use small-tier. For code output use mid-tier. For planning tasks with complex codebase use large-tier.

PASS Full pass. Cost and latency arguments correct. Tier breakdown is practical and sound.

COMPLETE ANSWER

The cleaner version of the decision rule: start mid-tier in development. Once it works, test small. If quality holds, ship small. Only escalate to large when mid-tier demonstrably fails on your specific task.

The critical word is 'fails' — not 'might do better' or 'feels safer'. Large models cost 30 to 150x more than small ones. That cost is only justified when there is a measurable quality gap on your actual task.

Q5. Why might the 8B model extract a wrong field where the 70B does not? (Not attempted)

YOUR ANSWER

Not attempted.

PARTIAL PASS Answer provided below for you to internalise before Session 2.

COMPLETE ANSWER

A larger model has more parameters, encoding more patterns from training. For any given token prediction, its probability distribution is more peaked — the correct answer has a much higher probability relative to wrong alternatives.

An 8B model has a flatter distribution on the same task. The 70B might produce: INR=94%, rupees=3%, Rs=2%. The 8B might produce: INR=51%, rupees=31%, Rs=12%. At temperature 0, both pick INR. Both are correct.

Now change the input slightly — an unusual phrasing or an edge case. The 8B distribution might tip: rupees=48%, INR=41%. At temperature 0, the 8B now picks rupees — wrong. Temperature had nothing to do with it. The model simply was not confident enough in the right answer.

This is why temperature 0 does not fully protect you from wrong answers. It removes sampling randomness. It does not add knowledge the model does not have.

Q6. Follow-up: What does 'temperature 0 removes sampling randomness but not model uncertainty' mean?

QUESTION ASKED IN CHAT — STRONG ENGAGEMENT

You asked this question after reviewing Q5. Asking it means you noticed the gap in your mental model and pushed to close it. That is exactly the right instinct.

Two separate sources of wrong answers

There are two completely different reasons a model can give you a wrong answer. Temperature controls only one of them.

Source 1: Sampling randomness (temperature controls this)

Imagine the model has calculated its distribution for the next token:

`"INR"`	72%
`"rupees"`	18%
`"Rs"`	7%
`"dollars"`	3%

At temperature 1, the model samples from this distribution — 28% of the time it picks something other than INR. That is sampling randomness.

At temperature 0, the model always picks INR — the top token. Sampling randomness is now zero. You have eliminated this source of error.

Source 2: Model uncertainty (temperature cannot touch this)

Model uncertainty is about what the probability distribution looks like before any sampling happens. Compare the same task across two model sizes:

Token	70B model	8B model
`"INR"`	94%	51%
`"rupees"`	3%	31%
`"Rs"`	2%	12%
`"dollars"`	1%	6%

At temperature 0 both models pick INR — both are correct on this input. But the 8B was only 51% confident vs the 70B at 94%. Now use a slightly unusual phrasing or an edge case. The 8B distribution might tip to: rupees=48%, INR=41%. At temperature 0 the 8B now picks rupees — wrong. Temperature had nothing to do with it.

THE DANGEROUS CONSEQUENCE

Temperature 0 gives you the model's best guess, consistently, every time. But if the model's best guess is wrong, temperature 0 gives you that wrong answer consistently and confidently — forever. At temperature 1 at least the wrong answer varies, so you might notice the instability. At temperature 0 a wrong answer can silently corrupt your data at scale before anyone catches it.

This is why model selection matters as much as temperature selection. Temperature 0 removes luck. It does not add knowledge the model does not have.

Session 1 — What to carry into Session 2

1	Transformers predict one token at a time by sampling from a probability distribution shaped by everything before it. The quality of that distribution is what separates a 70B from an 8B model.
2	Context window = working memory. RAG exists because most real documents do not fit. Chunking + retrieval = sending only the relevant slice.
3	Temperature 0 = deterministic. Always use it for structured outputs, classification, and extraction. Raise it only when variation is a feature.
4	Temperature 0 removes sampling randomness. It does not remove model uncertainty. A confidently wrong answer at temperature 0 is harder to catch than a visibly inconsistent answer at temperature 1.
5	Model tiers: small for classification and extraction, mid for 90% of production features, large only when mid demonstrably fails.
6	Streaming = connection stays open while tokens arrive. res.json() closes immediately. Session 7 wires this up in NestJS properly.

Session 2 builds directly on this. You will take the system prompt from Step 3 of Session 1 and turn it into a production-grade structured output template — reliable JSON, every time, regardless of how the user phrases their input. The temperature and model choices you made in Q2 will be the defaults you use throughout.